Arquitectura simplificada para alertas de fraude

Implementacion detallada en Spark SQL + Hive Metastore

Flujo principal

```
Snapshots de alertas
        |
        v
Deduplicacion de snapshots
        |
        v
Primera alerta valida por party_id
        |
        v
Personas con alerta previa o activa
        |
        v
Observaciones mensuales anteriores a la alerta
        |
        v
Labels de 1, 3, 6 y 12 meses
```

Documento de trabajo - agosto de 2026

Resumen ejecutivo

El objetivo es construir un dataset longitudinal, a nivel de persona y mes, que permita evaluar si una persona registra su primera alerta observada dentro de los siguientes 1, 3, 6 o 12 meses. La tabla de alertas contiene snapshots repetidos de la misma alerta, ya que una alerta permanece activa durante siete anos y el mismo party_id no puede colocar una nueva alerta durante ese periodo.

Definicion del outcome: El label representa la creacion de una alerta, no una victimizacion por fraude confirmada.

Restriccion critica: Para generar negativos poblacionales validos se necesita una fuente adicional con el universo mensual de personas elegibles, incluyendo quienes nunca tuvieron una alerta.

Fuentes requeridas

Fuente 1: tabla de alertas existente

```
source_db.security_alert
```

Columnas esperadas:

```
party_id
sec_alert_typ_num
crat_dte
expire_dte
upd_tsp
data_date
crat_dte_age
expire_dte_age
upd_tsp_age
upd_tsp_days
```

Fuente 2: universo mensual

```
source_db.party_month_source
```

Este nombre es provisional. Debe sustituirse por una tabla real que incluya personas con y sin alerta y, como minimo, party_id, data_date y active_flag. No debe construirse solamente desde security_alert.

Paso 0. Configuracion de Spark y base de trabajo

No se fija todavia spark.sql.shuffle.partitions porque debe ajustarse al volumen, numero de executors, cores, tamano de archivos y skew. Se habilita Adaptive Query Execution.

```
SET spark.sql.adaptive.enabled = true;
SET spark.sql.adaptive.coalescePartitions.enabled = true;
SET spark.sql.adaptive.skewJoin.enabled = true;

SET spark.sql.sources.partitionOverwriteMode = dynamic;

CREATE DATABASE IF NOT EXISTS fraud_lab;
```

Paso 1. Parametros del estudio

Parametros iniciales: periodo 2022-2025, duracion de alerta de 84 meses, historial minimo de 6 meses, cobertura minima de 80% y gap operacional inicial de 0 meses.

```
DROP TABLE IF EXISTS fraud_lab.cfg_project;

CREATE TABLE fraud_lab.cfg_project
USING PARQUET
AS
SELECT
    CAST('2022-01-01' AS DATE) AS study_start_date,
    CAST('2025-12-31' AS DATE) AS study_end_date,

    84 AS alert_duration_months,
    6 AS minimum_history_months,
    CAST(0.80 AS DOUBLE) AS minimum_history_coverage,
    0 AS gap_months;
```

```
SELECT *
FROM fraud_lab.cfg_project;
```

Paso 2. Configurar tipos de alerta validos

La configuracion es provisional. Los tipos usados como outcome deben ser confirmados por el dueno del dato.

```
DROP TABLE IF EXISTS fraud_lab.cfg_valid_alert_type;

CREATE TABLE fraud_lab.cfg_valid_alert_type
USING PARQUET
AS
SELECT *
FROM VALUES
    (1, 'ACTIVE_DUTY_ALERT', 0),
    (2, 'INITIAL_FRAUD_ALERT', 1),
    (3, 'EXTENDED_FRAUD_ALERT', 1),
    (4, 'CALIFORNIA_ALERT', 0),
    (5, 'LOUISIANA_ALERT', 0),
    (6, 'TEXAS_ALERT', 0)
AS config(
    sec_alert_typ_num,
    alert_type_description,
    use_as_outcome
);
```

```
SELECT *
FROM fraud_lab.cfg_valid_alert_type
ORDER BY sec_alert_typ_num;
```

Paso 3. Crear una tabla reducida de snapshots

Se crea una version angosta y particionada por mes. No se filtra crat_dte desde 2022, porque una alerta anterior puede seguir activa durante el periodo de estudio.

```
DROP TABLE IF EXISTS fraud_lab.alert_snapshot_raw;

CREATE TABLE fraud_lab.alert_snapshot_raw
USING PARQUET
PARTITIONED BY (snapshot_yyyymm)
AS
SELECT /*+ BROADCAST(t) */
    s.party_id,

    CAST(s.sec_alert_typ_num AS INT)
        AS sec_alert_typ_num,

    CAST(s.crat_dte AS DATE)
        AS alert_create_date,

    CAST(s.expire_dte AS DATE)
        AS alert_expire_date,
```

```
    CAST(s.upd_tsp AS TIMESTAMP)
        AS alert_update_timestamp,

    CAST(s.data_date AS DATE)
        AS snapshot_date,

    CAST(s.crat_dte_age AS INT)
        AS alert_age_months_source,

    CAST(s.expire_dte_age AS INT)
        AS months_to_expiration_source,

    CAST(s.upd_tsp_age AS INT)
        AS update_age_months_source,

    CAST(s.upd_tsp_days AS INT)
        AS update_age_days_source,

    year(CAST(s.data_date AS DATE)) * 100
        + month(CAST(s.data_date AS DATE))
        AS snapshot_yyyymm

FROM source_db.security_alert AS s

INNER JOIN fraud_lab.cfg_valid_alert_type AS t
    ON CAST(s.sec_alert_typ_num AS INT)
       = t.sec_alert_typ_num
   AND t.use_as_outcome = 1

CROSS JOIN fraud_lab.cfg_project AS c

WHERE s.party_id IS NOT NULL
  AND s.crat_dte IS NOT NULL
  AND s.data_date IS NOT NULL

  AND CAST(s.data_date AS DATE)
      BETWEEN c.study_start_date
          AND c.study_end_date

  AND CAST(s.crat_dte AS DATE)
      <= c.study_end_date;
```

Validacion de volumen mensual

```
SELECT
    snapshot_yyyymm,
    COUNT(*) AS snapshot_rows,

    approx_count_distinct(party_id)
        AS approximate_parties,

    MIN(alert_create_date)
        AS minimum_alert_create_date,

    MAX(alert_create_date)
        AS maximum_alert_create_date

FROM fraud_lab.alert_snapshot_raw

GROUP BY snapshot_yyyymm

ORDER BY snapshot_yyyymm;
```

Validacion de fechas

```
SELECT
    SUM(
        CASE
            WHEN alert_create_date > snapshot_date
            THEN 1 ELSE 0
        END
    ) AS create_date_after_snapshot_count,
```

```
    SUM(
        CASE
            WHEN alert_expire_date < alert_create_date
            THEN 1 ELSE 0
        END
    ) AS expiration_before_creation_count,

    SUM(
        CASE
            WHEN alert_expire_date IS NULL
            THEN 1 ELSE 0
        END
    ) AS null_expiration_count

FROM fraud_lab.alert_snapshot_raw;
```

Paso 4. Validar la duracion de siete anos

La expiracion sirve para control de calidad e identificacion de alertas activas. No se usa para definir el label.

```
SELECT
    CASE
        WHEN alert_expire_date IS NULL
            THEN 'NULL_EXPIRATION'

        WHEN alert_expire_date
             = add_months(alert_create_date, 84)
            THEN 'EXACT_84_MONTHS'

        WHEN abs(
            datediff(
                alert_expire_date,
                add_months(alert_create_date, 84)
            )
        ) <= 31
            THEN 'APPROXIMATE_84_MONTHS'

        ELSE 'DIFFERENT_DURATION'
    END AS duration_validation,

    COUNT(*) AS row_count,

    approx_count_distinct(party_id)
        AS approximate_party_count

FROM fraud_lab.alert_snapshot_raw

GROUP BY
    CASE
        WHEN alert_expire_date IS NULL
            THEN 'NULL_EXPIRATION'

        WHEN alert_expire_date
             = add_months(alert_create_date, 84)
            THEN 'EXACT_84_MONTHS'

        WHEN abs(
            datediff(
                alert_expire_date,
                add_months(alert_create_date, 84)
            )
        ) <= 31
            THEN 'APPROXIMATE_84_MONTHS'

        ELSE 'DIFFERENT_DURATION'
    END

ORDER BY row_count DESC;
```

Paso 5. Deduplicar snapshots

Se eliminan duplicados dentro de cada snapshot. La clave provisional combina party_id, tipo, fecha de creacion y fecha de snapshot.

```
DROP TABLE IF EXISTS fraud_lab.alert_snapshot_deduplicated;

CREATE TABLE fraud_lab.alert_snapshot_deduplicated
USING PARQUET
PARTITIONED BY (snapshot_yyyymm)
AS
WITH ranked_snapshots AS
(
    SELECT
        r.*,

        ROW_NUMBER() OVER
        (
            PARTITION BY
                party_id,
                sec_alert_typ_num,
                alert_create_date,
                snapshot_date

            ORDER BY
                alert_update_timestamp DESC NULLS LAST,
                alert_expire_date DESC NULLS LAST
        ) AS snapshot_row_number

    FROM fraud_lab.alert_snapshot_raw AS r
)

SELECT
    party_id,
    sec_alert_typ_num,
    alert_create_date,
    alert_expire_date,
    alert_update_timestamp,
    snapshot_date,

    alert_age_months_source,
    months_to_expiration_source,
    update_age_months_source,
    update_age_days_source,

    snapshot_yyyymm

FROM ranked_snapshots

WHERE snapshot_row_number = 1;
```

Medir reduccion

```
SELECT
    'RAW_SNAPSHOT_ROWS' AS metric,
    COUNT(*) AS metric_value

FROM fraud_lab.alert_snapshot_raw

UNION ALL

SELECT
    'DEDUPLICATED_SNAPSHOT_ROWS',
    COUNT(*)

FROM fraud_lab.alert_snapshot_deduplicated;
```

Validar unicidad

```
SELECT
    party_id,
    sec_alert_typ_num,
    alert_create_date,
```

```
    snapshot_date,
    COUNT(*) AS duplicate_count

FROM fraud_lab.alert_snapshot_deduplicated

GROUP BY
    party_id,
    sec_alert_typ_num,
    alert_create_date,
    snapshot_date

HAVING COUNT(*) > 1

LIMIT 100;
```

Paso 6. Consolidar snapshots en fechas de creacion

Se reduce cada fecha de creacion a una fila por party_id. Una misma fecha puede tener varios tipos por actualizaciones administrativas.

```
DROP TABLE IF EXISTS fraud_lab.alert_creation_candidate;

CREATE TABLE fraud_lab.alert_creation_candidate
USING PARQUET
PARTITIONED BY (alert_create_year)
AS
SELECT
    party_id,
    alert_create_date,

    sort_array(
        collect_set(sec_alert_typ_num)
    ) AS observed_alert_types,

    MIN(snapshot_date)
        AS first_snapshot_date,

    MAX(snapshot_date)
        AS last_snapshot_date,

    MIN(alert_expire_date)
        AS minimum_expire_date,

    MAX(alert_expire_date)
        AS maximum_expire_date,

    MAX(alert_update_timestamp)
        AS latest_update_timestamp,

    COUNT(*) AS snapshot_count,

    COUNT(DISTINCT snapshot_yyyymm)
        AS observed_snapshot_month_count,

    add_months(alert_create_date, 84)
        AS expected_expire_date,

    year(alert_create_date)
        AS alert_create_year

FROM fraud_lab.alert_snapshot_deduplicated

GROUP BY
    party_id,
    alert_create_date;
```

```
SELECT
    party_id,
    alert_create_date,
    COUNT(*) AS duplicate_count
```

```
FROM fraud_lab.alert_creation_candidate

GROUP BY
    party_id,
    alert_create_date

HAVING COUNT(*) > 1

LIMIT 100;
```

Paso 7. Identificar multiples fechas de creacion

Por regla de negocio, una persona no puede colocar otra alerta dentro de los siguientes 84 meses. Cualquier fecha adicional dentro de ese periodo debe revisarse como posible actualizacion administrativa, error o excepcion.

```
DROP TABLE IF EXISTS fraud_lab.alert_creation_sequence;

CREATE TABLE fraud_lab.alert_creation_sequence
USING PARQUET
AS
WITH ordered_creations AS
(
    SELECT
        a.*,

        LAG(alert_create_date)
        OVER
        (
            PARTITION BY party_id
            ORDER BY alert_create_date
        ) AS previous_create_date

    FROM fraud_lab.alert_creation_candidate AS a
)

SELECT
    *,

    CASE
        WHEN previous_create_date IS NULL
            THEN 'FIRST_OBSERVED_ALERT'

        WHEN alert_create_date
             < add_months(previous_create_date, 84)
            THEN 'MULTIPLE_CREATE_DATE_WITHIN_7Y'

        ELSE 'POSSIBLE_NEW_ALERT_AFTER_7Y'
    END AS creation_sequence_status,

    CASE
        WHEN previous_create_date IS NULL
            THEN NULL

        ELSE FLOOR(
            months_between(
                alert_create_date,
                previous_create_date
            )
        )
    END AS months_since_previous_create

FROM ordered_creations;
```

```
SELECT
    creation_sequence_status,
    COUNT(*) AS creation_count,
    approx_count_distinct(party_id)
        AS approximate_party_count

FROM fraud_lab.alert_creation_sequence

GROUP BY creation_sequence_status
```

```
ORDER BY creation_count DESC;
```

```
SELECT
    party_id,
    previous_create_date,
    alert_create_date,
    months_since_previous_create,
    creation_sequence_status,
    observed_alert_types

FROM fraud_lab.alert_creation_sequence

WHERE creation_sequence_status
      = 'MULTIPLE_CREATE_DATE_WITHIN_7Y'

ORDER BY
    party_id,
    alert_create_date

LIMIT 200;
```

Paso 8. Crear la primera alerta valida por party_id

```
DROP TABLE IF EXISTS fraud_lab.party_first_alert;

CREATE TABLE fraud_lab.party_first_alert
USING PARQUET
AS
WITH ranked_alerts AS
(
    SELECT
        a.*,

        ROW_NUMBER() OVER
        (
            PARTITION BY party_id
            ORDER BY
                alert_create_date,
                first_snapshot_date
        ) AS alert_order

    FROM fraud_lab.alert_creation_candidate AS a
)

SELECT
    party_id,

    alert_create_date
        AS first_valid_alert_date,

    first_snapshot_date
        AS first_alert_snapshot_date,

    last_snapshot_date
        AS last_alert_snapshot_date,

    COALESCE(
        maximum_expire_date,
        add_months(alert_create_date, 84)
    ) AS first_alert_expire_date,

    observed_alert_types
        AS first_alert_type_set,

    snapshot_count
        AS first_alert_snapshot_count

FROM ranked_alerts

WHERE alert_order = 1;
```

```
SELECT
    party_id,
    COUNT(*) AS row_count

FROM fraud_lab.party_first_alert

GROUP BY party_id

HAVING COUNT(*) > 1

LIMIT 100;
```

Paso 9. Identificar personas con alerta previa o activa

Una alerta previa se creo antes del inicio del estudio. Una alerta activa al inicio cumple que la fecha de creacion es anterior o igual al inicio y la expiracion es posterior o igual.

```
DROP TABLE IF EXISTS fraud_lab.party_alert_status;

CREATE TABLE fraud_lab.party_alert_status
USING PARQUET
AS
SELECT /*+ BROADCAST(c) */
    a.party_id,

    a.first_valid_alert_date,
    a.first_alert_expire_date,
    a.first_alert_snapshot_date,
    a.last_alert_snapshot_date,
    a.first_alert_type_set,

    CASE
        WHEN a.first_valid_alert_date
             < c.study_start_date
        THEN 1
        ELSE 0
    END AS pre_study_alert_flag,

    CASE
        WHEN a.first_valid_alert_date
             <= c.study_start_date

         AND a.first_alert_expire_date
             >= c.study_start_date
        THEN 1
        ELSE 0
    END AS active_alert_at_study_start_flag,

    CASE
        WHEN a.first_valid_alert_date
             BETWEEN c.study_start_date
                 AND c.study_end_date

        THEN a.first_valid_alert_date
        ELSE NULL
    END AS first_alert_in_study_date,

    CASE
        WHEN a.first_valid_alert_date
             < c.study_start_date
        THEN 0
        ELSE 1
    END AS eligible_for_first_alert_model_flag

FROM fraud_lab.party_first_alert AS a

CROSS JOIN fraud_lab.cfg_project AS c;
```

```
SELECT
    pre_study_alert_flag,
```

```
    active_alert_at_study_start_flag,
    eligible_for_first_alert_model_flag,

    COUNT(*) AS party_count

FROM fraud_lab.party_alert_status

GROUP BY
    pre_study_alert_flag,
    active_alert_at_study_start_flag,
    eligible_for_first_alert_model_flag

ORDER BY
    pre_study_alert_flag,
    active_alert_at_study_start_flag;
```

Regla de modelado: Las personas con alerta previa se excluyen del modelo de primera alerta.

Paso 10. Construir el universo mensual

Requisito indispensable: Este paso necesita una fuente historica con personas con y sin alerta. No debe utilizarse security_alert como universo.

Nombre provisional:

```
source_db.party_month_source
```

```
DROP TABLE IF EXISTS fraud_lab.party_month_universe;

CREATE TABLE fraud_lab.party_month_universe
USING PARQUET
PARTITIONED BY (observation_yyyymm)
AS
SELECT
    p.party_id,

    trunc(
        CAST(p.data_date AS DATE),
        'MM'
    ) AS observation_month,

    MAX(
        CAST(p.data_date AS DATE)
    ) AS latest_source_date_in_month,

    MAX(
        CASE
            WHEN p.active_flag = 1
            THEN 1
            ELSE 0
        END
    ) AS active_flag,

    year(CAST(p.data_date AS DATE)) * 100
        + month(CAST(p.data_date AS DATE))
        AS observation_yyyymm

FROM source_db.party_month_source AS p

CROSS JOIN fraud_lab.cfg_project AS c

WHERE p.party_id IS NOT NULL
  AND p.data_date IS NOT NULL

  AND CAST(p.data_date AS DATE)
      BETWEEN c.study_start_date
```

```
          AND c.study_end_date

GROUP BY
    p.party_id,

    trunc(
        CAST(p.data_date AS DATE),
        'MM'
    ),

    year(CAST(p.data_date AS DATE)) * 100
        + month(CAST(p.data_date AS DATE));
```

Validar unicidad mensual

```
SELECT
    party_id,
    observation_month,
    COUNT(*) AS row_count

FROM fraud_lab.party_month_universe

GROUP BY
    party_id,
    observation_month

HAVING COUNT(*) > 1

LIMIT 100;
```

Volumen mensual

```
SELECT
    observation_yyyymm,
    COUNT(*) AS party_month_rows,

    approx_count_distinct(party_id)
        AS approximate_party_count,

    SUM(active_flag)
        AS active_party_rows

FROM fraud_lab.party_month_universe

GROUP BY observation_yyyymm

ORDER BY observation_yyyymm;
```

Paso 11. Calcular el historial observado

Primero se crea un resumen por persona para evitar repetir ventanas costosas. Luego se calcula tenure y cobertura para cada mes.

```
DROP TABLE IF EXISTS fraud_lab.party_observation_summary;

CREATE TABLE fraud_lab.party_observation_summary
USING PARQUET
AS
SELECT
    party_id,

    MIN(observation_month)
        AS first_seen_month,

    MAX(observation_month)
        AS last_seen_month,
```

```
    COUNT(*) AS total_observed_month_count,

    FLOOR(
        months_between(
            MAX(observation_month),
            MIN(observation_month)
        )
    ) + 1 AS possible_observed_month_count,

    CAST(COUNT(*) AS DOUBLE)
    /
    NULLIF(
        FLOOR(
            months_between(
                MAX(observation_month),
                MIN(observation_month)
            )
        ) + 1,
        0
    ) AS total_history_coverage_ratio

FROM fraud_lab.party_month_universe

GROUP BY party_id;
```

```
DROP TABLE IF EXISTS fraud_lab.party_month_tenure;

CREATE TABLE fraud_lab.party_month_tenure
USING PARQUET
PARTITIONED BY (observation_yyyymm)
AS
WITH monthly_rank AS
(
    SELECT
        p.*,

        ROW_NUMBER() OVER
        (
            PARTITION BY p.party_id
            ORDER BY p.observation_month
        ) AS months_seen_to_date

    FROM fraud_lab.party_month_universe AS p
)

SELECT
    m.party_id,
    m.observation_month,
    m.latest_source_date_in_month,
    m.active_flag,

    s.first_seen_month,
    s.last_seen_month,

    CAST(
        FLOOR(
            months_between(
                m.observation_month,
                s.first_seen_month
            )
        ) AS INT
    ) AS observed_tenure_months,

    m.months_seen_to_date,

    CAST(m.months_seen_to_date AS DOUBLE)
    /
    CAST(
        FLOOR(
            months_between(
                m.observation_month,
                s.first_seen_month
            )
        ) + 1
        AS DOUBLE
    ) AS history_coverage_ratio,

    s.total_observed_month_count,
```

```
    s.total_history_coverage_ratio,

    m.observation_yyyymm

FROM monthly_rank AS m

INNER JOIN fraud_lab.party_observation_summary AS s
    ON m.party_id = s.party_id;
```

Validar tenure

```
SELECT
    CASE
        WHEN observed_tenure_months < 3
            THEN '00_02'

        WHEN observed_tenure_months < 6
            THEN '03_05'

        WHEN observed_tenure_months < 12
            THEN '06_11'

        WHEN observed_tenure_months < 24
            THEN '12_23'

        ELSE '24_PLUS'
    END AS tenure_band,

    COUNT(*) AS observation_count,

    approx_count_distinct(party_id)
        AS approximate_party_count,

    AVG(history_coverage_ratio)
        AS average_history_coverage

FROM fraud_lab.party_month_tenure

GROUP BY
    CASE
        WHEN observed_tenure_months < 3
            THEN '00_02'

        WHEN observed_tenure_months < 6
            THEN '03_05'

        WHEN observed_tenure_months < 12
            THEN '06_11'

        WHEN observed_tenure_months < 24
            THEN '12_23'

        ELSE '24_PLUS'
    END

ORDER BY tenure_band;
```

Paso 12. Crear observaciones mensuales anteriores a la alerta

Cada fila representa party_id + observation_month. La observacion del mismo mes de la alerta se excluye para garantizar que todas las features sean anteriores al evento.

```
DROP TABLE IF EXISTS fraud_lab.observation_base;

CREATE TABLE fraud_lab.observation_base
USING PARQUET
PARTITIONED BY (observation_yyyymm)
AS
SELECT
    m.party_id,
```

```
    m.observation_month,

    last_day(m.observation_month)
        AS feature_cutoff_date,

    add_months(m.observation_month, 1)
        AS score_date,

    m.first_seen_month,
    m.last_seen_month,
    m.observed_tenure_months,
    m.months_seen_to_date,
    m.history_coverage_ratio,
    m.total_history_coverage_ratio,

    COALESCE(
        a.pre_study_alert_flag,
        0
    ) AS pre_study_alert_flag,

    COALESCE(
        a.active_alert_at_study_start_flag,
        0
    ) AS active_alert_at_study_start_flag,

    a.first_alert_in_study_date,

    m.observation_yyyymm

FROM fraud_lab.party_month_tenure AS m

LEFT JOIN fraud_lab.party_alert_status AS a
    ON m.party_id = a.party_id

CROSS JOIN fraud_lab.cfg_project AS c

WHERE m.active_flag = 1

  AND m.observed_tenure_months
      >= c.minimum_history_months

  AND m.history_coverage_ratio
      >= c.minimum_history_coverage

  AND COALESCE(
        a.pre_study_alert_flag,
        0
      ) = 0

  AND COALESCE(
        a.active_alert_at_study_start_flag,
        0
      ) = 0

  AND
  (
      a.first_alert_in_study_date IS NULL

      OR m.observation_month
         < trunc(
             a.first_alert_in_study_date,
             'MM'
         )
  );
```

```
SELECT
    COUNT(*) AS invalid_observation_count

FROM fraud_lab.observation_base

WHERE first_alert_in_study_date IS NOT NULL

  AND observation_month
      >= trunc(
          first_alert_in_study_date,
          'MM'
```

```
      );
```

Paso 13. Crear labels de 1, 3, 6 y 12 meses

Los labels se almacenan en formato ancho para no multiplicar el volumen por cuatro. El valor 1 indica evento en la ventana; 0 indica no evento con seguimiento completo; NULL indica censura o seguimiento insuficiente.

```
DROP TABLE IF EXISTS fraud_lab.label_wide;

CREATE TABLE fraud_lab.label_wide
USING PARQUET
PARTITIONED BY (observation_yyyymm)
AS
WITH label_dates AS
(
    SELECT
        o.*,

        add_months(
            o.score_date,
            c.gap_months
        ) AS label_start_date,

        add_months(
            add_months(
                o.score_date,
                c.gap_months
            ),
            1
        ) AS label_end_1m_exclusive,

        add_months(
            add_months(
                o.score_date,
                c.gap_months
            ),
            3
        ) AS label_end_3m_exclusive,

        add_months(
            add_months(
                o.score_date,
                c.gap_months
            ),
            6
        ) AS label_end_6m_exclusive,

        add_months(
            add_months(
                o.score_date,
                c.gap_months
            ),
            12
        ) AS label_end_12m_exclusive,

        c.study_end_date,
        c.gap_months

    FROM fraud_lab.observation_base AS o

    CROSS JOIN fraud_lab.cfg_project AS c
),

followup_flags AS
(
    SELECT
        d.*,

        CASE
            WHEN first_alert_in_study_date
                 >= score_date

             AND first_alert_in_study_date
                 < label_start_date
```

```
            THEN 1
            ELSE 0
        END AS event_in_gap_flag,

        CASE
            WHEN study_end_date
                 >= date_sub(label_end_1m_exclusive, 1)

             AND last_seen_month
                 >= trunc(
                     date_sub(
                         label_end_1m_exclusive,
                         1
                     ),
                     'MM'
                 )
            THEN 1
            ELSE 0
        END AS complete_followup_1m_flag,

        CASE
            WHEN study_end_date
                 >= date_sub(label_end_3m_exclusive, 1)

             AND last_seen_month
                 >= trunc(
                     date_sub(
                         label_end_3m_exclusive,
                         1
                     ),
                     'MM'
                 )
            THEN 1
            ELSE 0
        END AS complete_followup_3m_flag,

        CASE
            WHEN study_end_date
                 >= date_sub(label_end_6m_exclusive, 1)

             AND last_seen_month
                 >= trunc(
                     date_sub(
                         label_end_6m_exclusive,
                         1
                     ),
                     'MM'
                 )
            THEN 1
            ELSE 0
        END AS complete_followup_6m_flag,

        CASE
            WHEN study_end_date
                 >= date_sub(label_end_12m_exclusive, 1)

             AND last_seen_month
                 >= trunc(
                     date_sub(
                         label_end_12m_exclusive,
                         1
                     ),
                     'MM'
                 )
            THEN 1
            ELSE 0
        END AS complete_followup_12m_flag

    FROM label_dates AS d
)

SELECT
    f.*,

    CASE
        WHEN event_in_gap_flag = 1
            THEN CAST(NULL AS INT)

        WHEN first_alert_in_study_date
             >= label_start_date
```

```
         AND first_alert_in_study_date
             < label_end_1m_exclusive
            THEN 1

        WHEN complete_followup_1m_flag = 1
            THEN 0

        ELSE CAST(NULL AS INT)
    END AS alert_next_1m,

    CASE
        WHEN event_in_gap_flag = 1
            THEN CAST(NULL AS INT)

        WHEN first_alert_in_study_date
             >= label_start_date

         AND first_alert_in_study_date
             < label_end_3m_exclusive
            THEN 1

        WHEN complete_followup_3m_flag = 1
            THEN 0

        ELSE CAST(NULL AS INT)
    END AS alert_next_3m,

    CASE
        WHEN event_in_gap_flag = 1
            THEN CAST(NULL AS INT)

        WHEN first_alert_in_study_date
             >= label_start_date

         AND first_alert_in_study_date
             < label_end_6m_exclusive
            THEN 1

        WHEN complete_followup_6m_flag = 1
            THEN 0

        ELSE CAST(NULL AS INT)
    END AS alert_next_6m,

    CASE
        WHEN event_in_gap_flag = 1
            THEN CAST(NULL AS INT)

        WHEN first_alert_in_study_date
             >= label_start_date

         AND first_alert_in_study_date
             < label_end_12m_exclusive
            THEN 1

        WHEN complete_followup_12m_flag = 1
            THEN 0

        ELSE CAST(NULL AS INT)
    END AS alert_next_12m

FROM followup_flags AS f;
```

Paso 14. Validar los labels

Resumen por horizonte

```
WITH stacked_labels AS
(
    SELECT
```

```
        party_id,
        observation_month,
        1 AS horizon_months,
        alert_next_1m AS label_value
    FROM fraud_lab.label_wide

    UNION ALL

    SELECT
        party_id,
        observation_month,
        3,
        alert_next_3m
    FROM fraud_lab.label_wide

    UNION ALL

    SELECT
        party_id,
        observation_month,
        6,
        alert_next_6m
    FROM fraud_lab.label_wide

    UNION ALL

    SELECT
        party_id,
        observation_month,
        12,
        alert_next_12m
    FROM fraud_lab.label_wide
)

SELECT
    horizon_months,

    COUNT(*) AS candidate_rows,

    SUM(
        CASE
            WHEN label_value IS NOT NULL
            THEN 1 ELSE 0
        END
    ) AS labeled_rows,

    SUM(
        CASE
            WHEN label_value = 1
            THEN 1 ELSE 0
        END
    ) AS positive_rows,

    SUM(
        CASE
            WHEN label_value = 0
            THEN 1 ELSE 0
        END
    ) AS negative_rows,

    SUM(
        CASE
            WHEN label_value IS NULL
            THEN 1 ELSE 0
        END
    ) AS censored_rows,

    approx_count_distinct(
        CASE
            WHEN label_value = 1
            THEN party_id
        END
    ) AS approximate_positive_parties,

    CAST(
        SUM(
            CASE
                WHEN label_value = 1
                THEN 1 ELSE 0
```

```
            END
        ) AS DOUBLE
    )
    /
    NULLIF(
        SUM(
            CASE
                WHEN label_value IS NOT NULL
                THEN 1 ELSE 0
            END
        ),
        0
    ) AS positive_rate

FROM stacked_labels

GROUP BY horizon_months

ORDER BY horizon_months;
```

Estabilidad anual

```
WITH stacked_labels AS
(
    SELECT
        party_id,
        observation_month,
        1 AS horizon_months,
        alert_next_1m AS label_value
    FROM fraud_lab.label_wide

    UNION ALL

    SELECT
        party_id,
        observation_month,
        3,
        alert_next_3m
    FROM fraud_lab.label_wide

    UNION ALL

    SELECT
        party_id,
        observation_month,
        6,
        alert_next_6m
    FROM fraud_lab.label_wide

    UNION ALL

    SELECT
        party_id,
        observation_month,
        12,
        alert_next_12m
    FROM fraud_lab.label_wide
)

SELECT
    horizon_months,
    year(observation_month) AS observation_year,

    SUM(
        CASE
            WHEN label_value IS NOT NULL
            THEN 1 ELSE 0
        END
    ) AS labeled_rows,

    SUM(
        CASE
            WHEN label_value = 1
            THEN 1 ELSE 0
        END
    ) AS positive_rows,
```

```
    approx_count_distinct(
        CASE
            WHEN label_value = 1
            THEN party_id
        END
    ) AS approximate_positive_parties,

    CAST(
        SUM(
            CASE
                WHEN label_value = 1
                THEN 1 ELSE 0
            END
        ) AS DOUBLE
    )
    /
    NULLIF(
        SUM(
            CASE
                WHEN label_value IS NOT NULL
                THEN 1 ELSE 0
            END
        ),
        0
    ) AS positive_rate

FROM stacked_labels

GROUP BY
    horizon_months,
    year(observation_month)

ORDER BY
    horizon_months,
    observation_year;
```

Resultado final de esta etapa

La tabla principal es fraud_lab.label_wide, con una fila por party_id + observation_month y las fechas de corte, tenure observado, seguimiento y labels de 1, 3, 6 y 12 meses.

```
security_alert
        |
        v
alert_snapshot_raw
        |
        v
alert_snapshot_deduplicated
        |
        v
alert_creation_candidate
        |
        v
party_first_alert
        |
        v
party_alert_status
        |
        v
party_month_universe
        |
        v
party_month_tenure
        |
        v
observation_base
        |
        v
label_wide
```

Punto de control: Antes de ejecutar los pasos 10 a 14 se debe identificar la tabla institucional que funcionara como source_db.party_month_source.

Recomendaciones para grandes volumenes

- Materializar las etapas costosas en tablas Parquet; no ejecutar toda la arquitectura como un unico CTE.
- Particionar por mes o ano, no por party_id.
- Mantener las tablas intermedias angostas hasta validar el label.
- Usar approx_count_distinct durante exploracion y conteos exactos solo para resultados finales.
- Aplicar BROADCAST unicamente a tablas de configuracion pequenas.
- Revisar EXPLAIN FORMATTED antes de joins grandes y confirmar partition pruning.
- Ejecutar ANALYZE TABLE ... COMPUTE STATISTICS NOSCAN despues de materializar tablas grandes.
- Revisar skew de party_id y distribucion de archivos pequenos antes de ajustar shuffle partitions.

Checklist de validacion

- Cero duplicados en party_id + sec_alert_typ_num + alert_create_date + snapshot_date despues de deduplicar.
- Una fila por party_id + alert_create_date en alert_creation_candidate.
- Una fila por party_id en party_first_alert.
- Las fechas adicionales dentro de siete anos quedan marcadas como anomalias.
- Las personas con alerta previa o activa al inicio quedan fuera del modelo de primera alerta.
- Una fila por party_id + observation_month en el universo mensual.
- Ninguna observacion ocurre en el mismo mes o despues de la primera alerta.
- Los labels con seguimiento incompleto son NULL, no 0.
- Las tasas positivas y la censura se revisan por horizonte y por ano.
- Los joins de atributos posteriores deben ser point-in-time y usar solo informacion anterior al feature_cutoff_date.